Объектно-ориентированное программирование

Лекция 4. Механизм исключений и перегрузка операций

Проблема обработки ошибок в процедурном стиле

```
57    GOptionContext* context = g_option_context_new(info);
58    if (!context) {
59        g_printerr("Failed to create GOptionContext.\n");
60        exit(EXIT_FAILURE);
61    }
62
63    g_option_context_add_main_entries(context, entries, NULL);
64
65    gchar* help = g_option_context_get_help(context, TRUE, NULL);
66    if (!help) {
67        g_printerr("Failed to get help message.\n");
68        g_option_context_free(context);
69        exit(EXIT_FAILURE);
70    }
71
72    GError* error = NULL;
73    if (!g_option_context_parse(context, argc, argv, &error)) {
74        g_printerr("%s\n\n%s", error->message, help);
75        g_free(help);
76        g_option_context_free(context);
77        exit(EXIT_FAILURE);
78    }
79
80    if (remaining == NULL || g_strv_length(remaining) != 3) {
81        g_printerr("Not all required arguments are specified.\n\n%s", help);
82        g_free(help);
83        g_option_context_free(context);
84        exit(EXIT_FAILURE);
85    }
86
87    const gchar* valid_modes[] = { "audio", "video", NULL };
88    if (!g_strv_contains(valid_modes, remaining[0])) {
89        g_printerr("Invalid mode '%s' (allowed values: 'audio', 'video').\n\n%s", remaining[0], help);
90        g_free(help);
91        g_option_context_free(context);
92        exit(EXIT_FAILURE);
93    }
```

Механизм исключений

```
#include <exception>

using namespace std;

void subop(int n) {
  …
  if (n == 1) {
    throw runtime_error("...");
  }
  …
}

void op() {
  subop(0);
  subop(1);
  subop(2);
}

int main() {
  op();
}
```

```
int main() {
  try {
    …
    op();
    …
  }
  catch (const runtime_error& e) {
    …
  }
}
```

Генерация исключения осуществляется с помощью ключевого слова `throw`.

В случае генерации исключения будет пропущен весь код, пока не будет встречен первый подходящий обработчик `catch`.

Если обработчик не будет найден и исключение покинет точку входа `main`, приложение упадёт.

Механизм исключений

```
try
  while (true)
    print(f.read_line())
catch
  print("EOF")
```

1. Исключения должны использоваться для ИСКЛЮЧИТЕЛЬНЫХ ситуаций
2. Исключения не должны использоваться для других целей, кроме как сообщения об исключительной ситуации
3. Исключения – дорогостоящий механизм в случае реальной генерации исключения

Гарантии безопасности исключений

Базовая гарантия
При возникновении любого исключения
состояние программы должно оставаться согласованным

Строгая гарантия
Если при выполнении операции возникает исключение,
то это не должно оказать какого-либо влияния на состояние приложения

Гарантия отсутствия исключений
Ни при каких обстоятельствах функция не будет генерировать исключения

Гарантии безопасности исключений

```
// Пример 1

void process(bool f) {
  auto data = new int[5];
  …
  if (f) {
    delete[] data;
    throw runtime_error("");
  }
  …
  delete[] data;
}
```

```
// Пример 2

void Circle::update() {
  radius = -1;

  if (f) {
    throw runtime_error("");
  }
  else {
    radius = 42;
  }
}
```

Гарантии безопасности исключений

```
// Пример 3

void Circle::update() {
  radius = -1;

  if (f) {
    radius = 0;
    throw runtime_error("");
  }
  else {
    radius = 42;
  }
}
```

```
// Пример 4

void Circle::update() {
  if (f) {
    throw runtime_error("");
  }
  else {
    radius = 42;
  }
}
```

```
// Пример 5

void Circle::update() {
  radius = 42;
}
```

Проблема многословности методов

```
struct Vector {
  float x;
  float y;

  Vector add(Vector rhs);

  Vector multiply(float f);
};

int main() {
  Vector a = …;
  Vector b = …;
  Vector c = …;

  auto r1 = a * 3 + b * 2 + c;
  auto r2 = a.multiply(3).add(b.multiply(2)).add(c);
}
```

Перегрузка операций

```
struct Vector {
  float x;
  float y;
};

// a += b
Vector& operator+=(Vector& a, const Vector& b) {
  a.x += b.x;
  a.y += b.y;
}

// a + b
Vector operator+(const Vector& a, const Vector& b) {
  Vector c(a);
  c += b;
  return c;
}

Vector& operator*=(Vector& a, float f);

Vector operator*(const Vector& a, float f);
```

Operator Overloading

Пример: ввод-вывод

```
#include <stdio.h>

FILE* stdin;
FILE* stdout;
FILE* stderr;

int fprintf(FILE* stream, const char* format, ...);

int fscanf(FILE* stream, const char* format, ...);
```

```
#include <iostream>

std::istream cin;
std::ostream cout;
std::ostream cerr;

std::istream& operator>>(std::istream& stream, T& item);

std::ostream& operator<<(std::ostream& stream, const T& item);
```

Пример: ввод-вывод

```
#include <iostream>

using namespace std;

int main() {
  ios::sync_with_stdio(false);

  int a = 0;
  int b = 0;
  cin >> a >> b;

  int sum = a + b;

  cout << a << " + " << b << " = " << sum;
}
```

Пример: ввод-вывод

```
#include <iostream>

using namespace std;

struct Complex {
  float re;
  float im;
  Complex() : re(0), im(0) { }
};

istream& operator>>(istream& stream, Complex& c) {
  stream >> c.re >> c.im;
  return stream;
}

ostream& operator<<(ostream& stream, const Complex& c) {
  stream << c.re << "+" << c.im << "j";
  return stream;
}
```

```
int main() {
  ios::sync_with_stdio(false);

  Complex c;
  cin >> c;
  cout << c;
}
```

Пример: строки

```
#include <iostream>
#include <string>

using namespace std;

int main() {
  ios::sync_with_stdio(false);

  string s("Hello");

  cout << s << endl;
  cout << s.starts_with("Hell") << endl;
  cout << s.starts_with("ello") << endl;
}
```

std::string